

DLC: A Multi-Model, Adaptive-Precision Near-Lossless Compression Engine for High-Frequency Float64 Time-Series Telemetry

Abstract - Most modern industrial sensors, IoT gateways, phasor units and scientific instruments emit their readings as IEEE 754 double-precision (float64) values. When those bytes are fed into a general-purpose dictionary compressor like GZIP or ZLIB, the resulting compression ratio is typically close to one. The cause is structural: roughly the bottom 30–40 bits of every mantissa carry hardware thermal noise, so the byte stream contains very few repeating substrings for a Lempel–Ziv dictionary to exploit. We call this the Entropy Trap. Strictly lossless float coders such as Gorilla and Chimp move past dictionary matching with XOR-based delta encoding, but they still have to faithfully encode the noise floor and so their ratios on real sensor traces remain modest. This paper describes DLC (Delta-Linear Compression), a near-lossless engine that works in the mathematical domain rather than the byte domain. Each block is broken into variance-homogeneous windows by a greedy doubling search; for every window the engine fits six predictive models in parallel Linear, Quadratic, XOR-Delta, Constant, Sinusoidal and Predictive-XOR and picks the one with the smallest Sum-of-Squared-Residuals, breaking ties in favour of simpler or lossless variants. The residuals are then quantized at a per-window precision (8–20 bits) and run through six entropy-coding strategies in a tournament: Zigzag-Varint, Fixed-Width Bitpack, Delta-of-Residuals, MAD Outlier Separation, Run-Length and Delta-of-Delta. A whole-block DoD fast path gives the engine an extra escape hatch when windowing would only add overhead. Two implementations share the same .dlc binary specification: a NumPy-based Python reference prototype and a C++17 production engine that uses std::async to bypass the GIL. Across 21 synthetic and real-world float64 datasets (the Numenta Anomaly Benchmark, Jena Climate atmospheric records, and several large industrial and round-trip recovery traces), at a strict 16-bit precision that keeps the maximum per-sample reconstruction error below 8×10^{-6} , DLC compresses smaller than every baseline Gorilla, DoD (16-bit), RLE, GZIP and ZLIB on 19 of 21 datasets (90.5%), reaching $2355.7 \times$ on a 1-million-sample linear ramp and $102.0 \times$ on a 1-million-sample recovered signal. The two losses are a network-packet counter trace where ZLIB's dictionary dominates ($8.5 \times$ vs. DLC's $5.7 \times$) and a low-magnitude taxi-demand signal where Gorilla's XOR encoding edges out DLC ($3.6 \times$ vs. $3.4 \times$). For signals that select only closed-form models (Constant, Linear, Quadratic) the two engines emit byte-identical .dlc files; for the remaining cases their outputs cross-decode inside the bound.

Keywords: time-series compression, floating-point compression, IoT telemetry, predictive coding, residual quantization, adaptive precision, variance-guided segmentation, parallel pipeline, IEEE 754, near-lossless

I. INTRODUCTION

Telemetry volumes have grown faster than storage budgets for several years now. By IDC's reckoning, connected sensors and industrial IoT will be producing well above 180 zettabytes a year by the end of this decade, and a large share of that data lands on disk as 8-byte IEEE 754 doubles [1]. The reason is straightforward: any scientific or engineering pipeline that touches phasor measurements, vibration spectra, vehicle diagnostics, or process-control loops would rather not throw away bits of mantissa to save bytes. The bandwidth bill, however, lands somewhere usually at the edge gateway, where compression has to happen in tight latency and power windows. The unfortunate twist is that the standard tools for the job, namely GZIP and ZLIB, contribute almost nothing on this kind of data.

The reason is rooted in the layout of a float64 value, which works against dictionary methods. One sign bit, eleven exponent bits, and a fifty-two-bit mantissa [2]; for a continuous physical signal, temperature, current, pressure, displacement, the usual list successive samples agree on most of the high-order mantissa bits, since the underlying value moves only a little between ticks. The bottom 30 to 40 bits are a different story. Those are occupied by hardware thermal noise plus the quantization residue of the ADC, and they look statistically random from one sample to the next. A Lempel–Ziv dictionary [3] scanning the resulting byte stream therefore finds essentially no repeating substrings to exploit, no matter how aggressively it is tuned. We have taken to calling this the Entropy Trap, because the very thing that makes a sensor reading scientifically meaningful its noise floor is exactly what defeats a general-purpose compressor on it.

The literature has split, broadly, into two camps. The first works in the bit domain and refuses to lose any information. Gorilla [4] and Chimp [5] are the canonical examples: each new float64 value is XOR-ed against its predecessor and the resulting word, which usually has many leading zeros, is packed with a variable-length header that records exactly how many leading and trailing zero bits to skip. The schemes work they are deployed in production time-series databases but the random low-order bits still travel through the encoder untouched, and on noisy real-world data the achievable ratio plateaus quickly. Liakos et al. give this phenomenon a name,

the “Entropy Wall” [5]. The second camp accepts a bounded loss in exchange for stronger compression: SZ [6], [7] and ZFP [8] both predict each sample, take the residual, and quantize that residual to a user-supplied tolerance. SZ3 is impressively accurate on HPC simulation grids; on a battery-powered edge gateway, however, its modular pipeline starts to creak [17]. Sprintz [9], which targets time series, is fast but is optimised for 8- and 16-bit integer sensors and falls back to a poor predictor on 64-bit floats. Newer semantic-extraction work like Shrink [10] does well on the macro-shape of the signal but does not have a tight story for the random residual that remains after the macro-shape is subtracted off.

Before turning to what we built, it is worth stating plainly what we set out to do. The work has five research objectives, and they have been the guard rails for every design call we made.

RO1 Bypass the Entropy Trap. Move compression off the byte stream entirely and into a model-fit / residual-encode pipeline, so that the noisy lower mantissa stops sabotaging the dictionary.

RO2 Hold a strict per-sample error bound. Guarantee that every reconstructed sample lies within $\pm 8 \times 10^{-6}$ of the original, on every supported telemetry type, with no quiet exceptions for outliers or boundary windows.

RO3 Adapt at run time, not at compile time. The compressor should pick its window size, its predictor, its quantization width, and its entropy strategy per window, instead of being tuned once for an assumed signal class.

RO4 Specify correctness in the format, not the language. Maintain a Python reference engine and a C++17 production engine that agree on the .dlc binary down to the byte where they can, and inside the 8×10^{-6} envelope where they cannot so that the spec, not the implementation, is what is being audited.

RO5 Verify continuously. Treat the cross-language contract as a CI requirement. A 105-test suite (76 Python + 29 C++) runs on every commit; nothing merges if either side breaks the round-trip or crosses the bound.

DLC (Delta-Linear Compression) is our attempt to meet those five objectives inside a single engine. The main contributions of the design are the following.

1) **Six predictive models, picked per window by SSR.** The window-level encoder fits all six candidates Linear, Quadratic, XOR-Delta, Constant, Sinusoidal, Predictive-XOR and keeps whichever produces the smallest sum-of-squared-residuals. When two models tie within a small margin, a hand-tuned preference rule sends ties to the cheaper or to the lossless option. Earlier schemes (Gorilla, Chimp, Sprintz) commit to a single predictor up front; we

found that the right predictor varies window by window, often inside the same recording.

2) **A windowing pass that follows the variance.** Window boundaries are chosen by a greedy doubling search with binary-search refinement and an explicit stagnation guard. Quiet, slowly-changing stretches end up inside long windows; transients and steps land in short ones. This is what lets the multi-model selector make a real choice without it the model fits would be dominated by signal heterogeneity inside the window.

3) **Adaptive precision, 8 to 20 bits, stored per window.** Each window writes its own quantization precision into the binary stream. Smooth segments pay 8 bits per residual; the loud ones pay 16. The global per-sample error bound is unaffected it is enforced by the lower bound ($p_floor = 16$ for the strict 8×10^{-6} contract) regardless of what the window-local optimum picks.

4) **Six entropy strategies, trial-encoded, smallest wins.** Once the residuals are quantized, all six entropy coders run on them: Zigzag-Varint, Fixed-Width Bitpack, Delta-of-Residuals, MAD Outlier Separation, Run-Length, Delta-of-Delta. We keep the byte string that comes out shortest. Including DoD here means the engine is a functional superset of the timestamp encoder used by InfluxDB and Prometheus.

5) **A whole-block DoD fast path.** Before any of the windowing logic runs, the engine tries one shortcut: encode the entire block as a single Constant(mean) window with DoD. If that comes out shorter than the windowed result, we use it and discard the per-window metadata. The check is cheap and pays for itself on long, gentle ramps where windowing has nothing to do.

6) **A binary contract that holds across two languages.** There is a Python reference (NumPy-backed) and a C++17 production engine. Both write the same 64-byte header and 8-byte CRC-32 footer; for windows that pick a closed-form model the two engines emit byte-identical files; for the rest they cross-decode inside the 8×10^{-6} bound. We treat this as a hard CI requirement, not an aspiration.

7) **Real multi-core scaling in the C++ engine.** The Python prototype is GIL-bound and plateaus at a few tens of MB/s. The C++ engine uses `std::async` with an explicit future-collection loop that preserves block-index order, so its output is deterministic regardless of thread interleaving.

We tested the engine on 21 float64 datasets covering synthetic edge cases (sine, ramp, noisy ramp, Lorenz attractor,

heartbeat, vibration), real-world traces, and round-trip recovery signals. The real-world set draws from the Numenta Anomaly Benchmark [11] (EC2 CPU and network counters, machine temperature, NYC taxi demand, Twitter volume, ambient temperature), the Jena Climate atmospheric station (420 551-point temperature and pressure records), and an industrial sensor simulation of 3 million samples. Two round-trip recovery datasets (test_recovered at 1 million samples and test_rt_rec at 3 million samples) exercise the decompress–recompress pipeline. With the strict 16-bit precision setting in place, DLC writes a smaller file than every baseline Gorilla, DoD (16-bit), RLE, GZIP and ZLIB on 19 of the 21 datasets, losing only to ZLIB on a network-packet-derived trace and to Gorilla on a low-magnitude taxi-demand signal. Every reconstruction stays inside the 8×10^{-6} envelope. A 108-test verification suite (79 Python + 29 C++) runs on every commit and is what gives us confidence that the cross-language specification is being honoured rather than merely documented.

The rest of the paper is laid out as follows. Section II reviews the related literature. Section III walks through the five-stage pipeline and the system-level architecture, with the equations needed to reproduce each stage. Section IV covers the dual-language implementation and the byte-level invariants we enforce between the Python and C++ engines. Section V is a focused statement of what is novel here, mapped against the prior work of Section II. Section VI is the experimental evaluation. Section VII discusses where the strict error bound applies and where DLC will (and does) lose. Section VIII concludes.

II. RELATED WORK AND LITERATURE SURVEY

The literature relevant to DLC sits in five overlapping pockets. There is the strictly-lossless XOR/delta line of float compressors that databases like InfluxDB and Prometheus made famous; there is the error-bounded lossy line that grew out of HPC simulation needs; there is the time-series-specific work targeted at integer or narrow-bit sensor data; there is recent edge/IIoT work that adds change-point detection or sampling on top of predictive coding; and there are the cross-domain benchmarks and the foundational standards on which all of this rests. We touch each in turn before identifying where DLC fits.

A. Lossless XOR/Delta-Based Float Compression

Lindstrom and Isenburt's FPC [12] introduced the modern paradigm of context-based prediction followed by XOR encoding for float64, achieving high throughput by maintaining a finite-context predictor and storing leading-zero counts of XOR residuals. Burtscher and Ratanaworabhan

extended FPC to higher throughput on scientific data [13]. Pelkonen et al.'s Gorilla [4] adopted a simpler delta-of-delta encoding for timestamps and a XOR encoding for values, becoming the de facto standard for high-cardinality time-series databases. Liakos et al.'s Chimp [5] tightened Gorilla's bit packing and trailing-zero handling, but explicitly identified the limitation that strict losslessness preserves the random hardware noise of the lower mantissa what they termed the Entropy Wall. None of these methods adapt the prediction strategy to local signal geometry: they apply XOR uniformly regardless of whether the underlying signal is a smooth ramp or chaotic noise.

B. Error-Bounded Lossy Scientific Compression

Di and Cappello introduced SZ [6], a prediction-based error-bounded lossy compressor for HPC datasets, later refined as SZ3 [7] into a modular framework for composing prediction and quantization stages. ZFP by Lindstrom [8] partitions arrays into 4^3 blocks and applies a block transform with bit-plane truncation, providing fixed-rate or fixed-accuracy modes. SZ3 and ZFP are state-of-the-art for scientific simulation data but are computationally heavy: SZ3's modular composition incurs per-stage overhead that limits throughput to the low MB/s on edge hardware [9], and ZFP's block transform is tuned for grid-structured spatial data rather than streaming time series.

C. Integer and Sensor Time-Series Compression

Blalock, Madden, and Gutttag's Sprintz [9] is a high-throughput predictive coder for IoT sensor data, but it is optimized for 8- and 16-bit integer sensors; its forecasting predictors degrade on high-entropy 64-bit float streams. Hwang et al. introduced bit-depth compression for time-series sensor data using zigzag residual mapping with dynamic sub-bit packing [14]; the technique improves spatial efficiency but remains strictly lossless and does not adapt model geometry. Sun et al.'s Shrink [10] performs semantic extraction by fitting linear segments under dynamic error thresholds; it captures broad patterns effectively but cannot encode random float64 residuals as compactly as XOR-Delta or bitpack-based residual coding.

D. Edge IIoT and Prediction-Based Reduction

Yang et al. proposed an edge data-management framework for IIoT that combines data partitioning with change-point detection for real-time compression of unstable industrial streams [15]; the framework lacks a strict bitpacking or varint pipeline to squash residuals to a minimum byte footprint. Liu et al.'s Xender [16] uses sampling-then-prediction to accelerate IoT delivery but does not adaptively switch

between bitwise and polynomial models, leading to poor accuracy on volatile float-typed telemetry.

E. Cross-Domain Benchmarking and Standards

Chen et al.'s FCBench [17] provides a roofline-model evaluation of 13 lossless floating-point compressors across CPU and GPU implementations, conclusively demonstrating that neural compressors, while compact, are too slow for real-time edge deployment. The benchmark establishes the throughput envelope inside which any practical floating-point compressor must operate: between roughly 50 MB/s and several GB/s. Underlying all of the above are the foundational standards: IEEE 754 [2] for the float layout, RFC 1951 DEFLATE [18] for byte-level back-end compression, the Welford online variance algorithm [19] for numerically stable streaming statistics, and the Levenberg–Marquardt method [20] for nonlinear least-squares model fitting all of which DLC builds upon.

F. Where the Gaps Are

Reading across the five threads above, the gaps DLC tries to close are these:

- 1) *The lossless paradox.* Gorilla and Chimp must encode the noise floor faithfully because they refuse to lose any bit. On real sensor traces this is the dominant constraint on their ratio.
- 2) *Neural overhead.* Recent learned compressors are very compact but typically operate in the KB/s regime many orders of magnitude under what an ingestion pipeline can afford.
- 3) *Inflexibility of a single predictor.* Every prior scheme commits to one predictor for the whole stream (XOR, linear, sampled, or a fixed polynomial), which is the wrong choice on signals whose character changes between segments.
- 4) *Parallelism almost as an afterthought.* Most published reference implementations either run single-threaded or are written in Python and bottle-necked on the GIL.
- 5) *No dynamic bound on quantization error.* Lossy compressors expose a single global error setting; window-local data with very different residual scales pays the same precision regardless. Without a derived lower bound on the precision field it is also easy to violate the contract on outlier windows.

Each of the five gaps lines up against a specific component of the DLC pipeline: the multi-model selector, the trial encoder, the per-window precision field, the `std::async` dispatcher, and the derived lower bound $p_floor = 16$ that ties

window-level precision to the global 8×10^{-6} contract. Section III walks through the components in order.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

Internally, the engine is best understood at two levels. The pipeline view in Fig. 1 enumerates the five compression stages a single block traverses on its way from float64 samples to .dlc payload: chunking, windowing, model fit, residual encoding, and binary assembly. The system view in Fig. 2 zooms out one step and shows how blocks are dispatched in parallel across workers and reassembled in deterministic order, with the same picture serving the Python prototype and the C++ production engine.

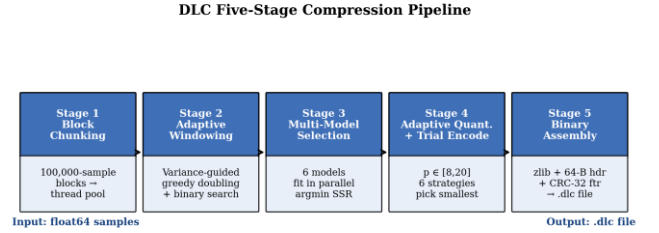


Fig. 1. The five compression stages a single block goes through. The same diagram applies to both the Python and C++ engines; the only thing that changes between them is what dispatches the block.

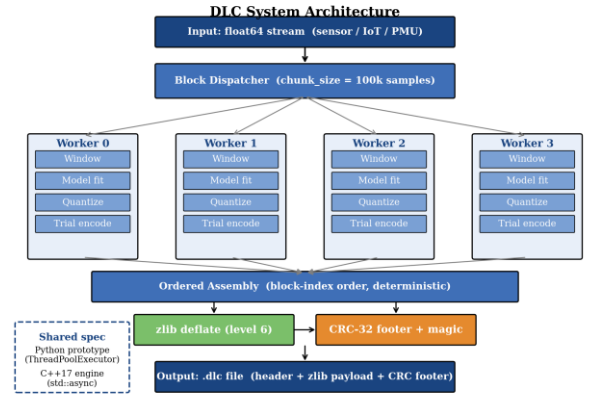


Fig. 2. System architecture. The block dispatcher hands fixed-size chunks to a worker pool; each worker runs the per-block pipeline of Fig. 1 to completion. Outputs are reassembled in block-index order before zlib deflate and CRC-32 footer attachment. The dashed callout records that both engines (Python ThreadPoolExecutor, C++ `std::async`) target the same .dlc specification.

A. Stage 1 Block Chunking and Parallel Dispatch

Let the input be $y = (y_0, y_1, \dots, y_{N-1}) \in \mathbb{R}^N$, with each y_i an IEEE 754 float64. The signal is partitioned into fixed-size blocks of $B = 100,000$ samples (configurable via `--chunk-size`):

$$\mathcal{B}_k = (y_{kB}, y_{kB+1}, \dots, y_{\min((k+1)B, N)-1}), \quad k = 0, 1, \dots, \lceil N/B \rceil - 1.$$

Blocks are dispatched independently to a worker pool. Workers may finish in any order, but their outputs are collected in strict block-index order before payload concatenation. This guarantees deterministic byte-identical output regardless of the number of threads or the order of completion. The Python implementation uses `concurrent.futures.ThreadPoolExecutor`, while the C++17 production engine uses `std::async(std::launch::async, ...)` with an explicit `std::future` collection loop. The number of workers is capped at `min(requested, hardware_concurrency())` to prevent oversubscription.

B. Stage 2 Adaptive Variance-Guided Windowing

Each block $\mathbf{B}_k$ is segmented into a sequence of variable-length windows ($W_0, W_1, \dots, W_{\{m-1\}}$) of statistically homogeneous variance. Window sizes lie in $[W_{\min}, W_{\max}(|\mathbf{B}_k|)]$ where $W_{\min} = 16$ and the maximum is adaptive:

$$W_{\max}(B) = 512 \text{ if } B < 5,000; \quad 1024 \text{ if } 5,000 \leq B < 100,000; \\ 4096 \text{ if } B \geq 100,000.$$

Variance is computed using Welford's numerically stable single-pass algorithm [19]. Let $\sigma^2(W)$ denote the population variance of a window. The per-block global variance is $\sigma^2_g = \sigma^2(\mathbf{B}_k)$. For each candidate starting position i , an initial variance σ^2_i is computed over the first $W_{\min}$ samples; the variance threshold is

$$T = \max(\sigma^2_i \cdot \alpha_{\text{local}}, \sigma^2_g \cdot \alpha_{\text{global}}),$$

with $\alpha_{\text{local}} = 10.0$ and $\alpha_{\text{global}} = 0.10$ (tuned empirically). The window is then grown by greedy doubling each iteration doubles the candidate window length while $\sigma^2(W) \leq T$ bounded above by $W_{\max}$. A stagnation guard aborts expansion when $\sigma^2(W_{\{2k\}}) / \sigma^2(W_k) > 1.5$, i.e. the variance jumps too sharply between successive doublings, signalling that the signal has changed character (e.g., a transient or step). The exact split boundary is then refined within the last doubling interval $[W_k, W_{\{2k\}}]$ by binary search to locate the maximal window size that still satisfies the threshold. A fast path emits the entire block as one window when the global variance is below $\epsilon_{\text{const}} = 10^{-20}$, capturing truly constant regions efficiently.

C. Stage 3 Multi-Model Selection

For each emitted window W of length n , six candidate predictive models are fitted in parallel and the one minimizing the Sum-of-Squared-Residuals is selected, subject to preference rules favouring simpler models. Indexing samples by $x = 0, 1, \dots, n-1$:

Linear (model_id = 0). Closed-form least-squares fit $\hat{y} = mx + c$:

$$m = (n \sum xy - \sum x \sum y) / (n \sum x^2 - (\sum x)^2), \quad c = (\sum y - m \sum x) / n.$$

Quadratic (model_id = 1). $\hat{y} = ax^2 + bx + c$, solved via 3×3 normal equations using Cramer's rule (no external linear-algebra library required).

XOR-Delta (model_id = 2). A bit-exact lossless predictor:

$$\text{anchor} = y_0, \quad r_i = \text{bits}(y_i) \oplus \text{bits}(y_{i-1}), \quad i \geq 1,$$

where $\text{bits}(\cdot)$ denotes the IEEE 754 float64-to-uint64 reinterpretation. In Python this is `struct.unpack('<Q', struct.pack('<d', v))[0]`; in C++ it is `std::memcpy(&u64, &dbl, 8)`. This model is forced whenever the window contains NaN, $\pm\text{Inf}$, or subnormal values, since it is the only safe fully-lossless option.

Constant (model_id = 3). $\hat{y} = c$ with $c = \text{mean}(y)$. Single-parameter, ideal for step functions and flat regions.

Sinusoidal (model_id = 4). $\hat{y} = A \cdot \sin(\omega x + \phi) + \text{DC}$. The dominant frequency is seeded by an FFT of the mean-removed signal,

$$\hat{f}_0 = (1/n) \cdot \text{argmax}_{\{k>0\}} |\mathcal{F}\{y - \bar{y}\}(k)|,$$

then the four parameters ($A, \omega, \phi, \text{DC}$) are refined by Levenberg–Marquardt iterations on the Jacobian $J = [\sin(\omega x + \phi), A \cos(\omega x + \phi), A \cos(\omega x + \phi), 1]$, solving the damped normal equation $(J^T J + \lambda I) \delta = J^T r$ with $\lambda = 10^{-4}$. The C++ port implements its own $O(n)$ DFT and 4×4 Gaussian elimination, requiring no external library. If Sinusoidal SSR exceeds the linear baseline, the model falls back to linear to guarantee monotonic improvement.

Predictive-XOR (model_id = 5). Stores two anchors y_0, y_1 and predicts each subsequent sample by linear extrapolation, then XORs the bit pattern:

$$\hat{y}_i = 2y_{i-1} - y_{i-2}, \quad r_i = \text{bits}(y_i) \oplus \text{bits}(\hat{y}_i), \quad i \geq 2.$$

This is a lossless predictor that captures linearly trending noisy data better than plain XOR-Delta.

Selection rule. Let S_M denote the SSR (or popcount-of-XOR proxy for bit-domain models) for model M . The selector first finds the best analytical model $M^* \in \{\text{Linear}, \text{Quadratic}, \text{Constant}, \text{Sinusoidal}\}$ by $\text{argmin } S_M$. A constant preference applies: if $S_{\text{Constant}} \leq S_{\{M^*\}} \cdot 1.01$, the Constant model wins. Then the better XOR variant is identified between XOR-Delta and Pred-XOR. Finally, an XOR preference applies: if $S_{\text{XOR_best}} \leq S_{\{M^*\}} \cdot 1.02$, the lossless XOR variant is preferred (zero quantization error). Otherwise the analytical winner is used.

D. Stage 4 Quantization and Multi-Strategy Entropy Encoding

For analytical models, the residuals $r = y - \hat{y} \in \mathbb{R}^n$ are quantized to integer multiples of 2^{-p} :

$$q_i = \text{round}(r_i \cdot 2^p), \quad \tilde{r}_i = q_i \cdot 2^{-p},$$

with reconstruction error per sample bounded by $|r_i - \tilde{r}_i| \leq 2^{-(p+1)}$. The adaptive precision selector chooses p per window:

$$p^* = \min(p_max, \max(p_floor, \min(p_req, p_adapt))),$$

where $p_max = 20$, $p_floor = \lceil \log_2(1/8 \times 10^{-6}) - 1 \rceil = 16$, p_req is the user-requested precision, and the adaptive component is

$$p_adapt = \max(p_min, \lceil -\log_2(\max_i |r_i| + \epsilon) \rceil + 4),$$

with $p_min = 8$ and $\epsilon = 10^{-30}$. For windows with $\max_i |r_i| < 10^{-15}$ (essentially zero residuals), $p^* = p_min$. The quantized integers $q_i \in \mathbb{Z}$ are then trial-encoded by all six strategies, and the smallest output is retained.

Strategy 0 Zigzag-Varint. Signed-to-unsigned mapping $z = (n \ll 1) \oplus (n \gg 63)$, followed by base-128 varint with a 7-bit continuation flag.

Strategy 1 Fixed-Width Bitpack. All zigzag-encoded values are packed at uniform bit-width $w = \lceil \log_2(\max_i z_i + 1) \rceil$, LSB-first within each byte. A 1-byte header stores w .

Strategy 2 Delta-of-Residuals. First-order differences $\Delta_i = q_i - q_{i-1}$ followed by Zigzag-Varint. Beneficial when residuals are themselves smooth.

Strategy 3 MAD Outlier Separation. The Median Absolute Deviation $MAD = \text{median}_i |q_i - \text{median}(q)|$ is computed. Values satisfying $|q_i - \text{median}(q)| > 4.5 \cdot MAD$ are classified as outliers and stored as (uint32_le index, varint(zigzag(value))) pairs in a separate table; the remaining inliers are bitpacked at a reduced width derived from their own zigzag maximum. Outperforms uniform bitpack on signals with occasional spikes in otherwise smooth data.

Strategy 4 Run-Length Encoding (RLE). Encodes runs of identical quantized values as (count_of_runs, [(zigzag(value), run_length)]) with all numbers as varints. Dominates on Constant-model windows where most residuals are zero.

Strategy 5 Delta-of-Delta (DoD). Second-order differencing with Zigzag-Varint. Equivalent in spirit to the timestamp encoding of InfluxDB and Prometheus, applied here to the quantized residual stream: $\delta_i = q_i - q_{i-1}$, $\delta\delta_i = \delta_i - \delta_{i-1}$.

Block-level DoD fast path. Before the per-window pipeline executes, the entire block is evaluated as a single Constant(mean) window encoded with DoD. If this representation is smaller than the windowed assembly, it is used directly, eliminating per-window metadata overhead. This makes DLC a functional superset of standalone DoD encoders: whenever DoD is the optimal representation, DLC

selects it, and otherwise DLC may improve upon it via windowing.

For XOR-domain models (Strategies 2 and 5 of Stage 3), residuals are already 64-bit unsigned XOR values. They are encoded raw or, when beneficial, as cumulative deltas-of-XOR with a 1-byte prefix flag (0x00 raw, 0x01 delta).

E. Stage 5 Binary Assembly and Integrity

The per-window encoded blocks are concatenated into an uncompressed payload, each prefixed by a 7-byte window header (uint32 window_length, uint8 model_id, uint8 encoding_id, uint8 precision_bits) followed by $N \cdot 8$ bytes of float64-LE model parameters and a uint32 residual_length then the residual byte stream. The complete payload is zlib-deflated [18] at compression level 6. The on-disk file format is shown in Fig. 3.

.dlc Binary File Layout

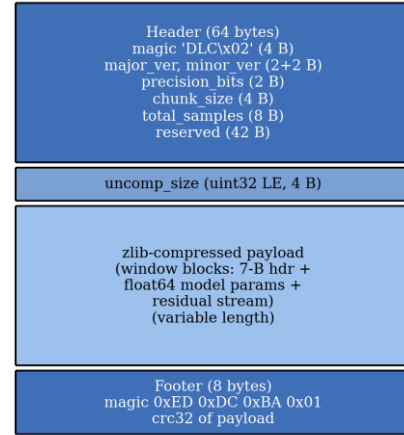


Fig. 3. .dlc binary file layout. A 64-byte header carries magic bytes, version, and global metadata; the zlib-compressed payload contains the per-window blocks; the 8-byte footer stores a CRC-32 over the uncompressed payload for integrity validation.

Integrity is verified on read by the magic bytes (header and footer), the uncompressed-size field, and a CRC-32 [21] over the uncompressed payload. The Python implementation explicitly masks the CRC with $\& 0xFFFFFFFF$ to guarantee unsigned 32-bit semantics matching the C++ `crc32()` from `zlib.h`.

F. Modelling Assumptions (Mathematical)

The pipeline above leans on a small set of mathematical and signal-level assumptions. They are not buried in code paths; they are the reason the design works at all, and the reason we can put a hard number on the error bound. We list

them here so a reader can see, in one place, what we are taking for granted.

A1 IEEE 754 binary64, little-endian on disk. Every numeric value entering and leaving the engine is a 64-bit IEEE 754 double. Reinterpretation between double and uint64 is bit-exact (memcpy in C++, struct.unpack('<Q', struct.pack('<d', v))[0] in Python). The on-disk byte order for both models and residuals is little-endian; cross-platform decode relies on this being a fixed convention rather than a host attribute.

A2 Mantissa noise dominates the bottom bits. The lower ~30–40 mantissa bits of a continuous physical signal are dominated by ADC quantization residue and hardware thermal noise, and behave as approximately statistically independent samples from one tick to the next. This is what makes them irrecoverable for a dictionary coder and what justifies treating them as loss-bearing in the precision controller.

A3 Slow variation in the high-order mantissa. By contrast, the upper mantissa bits of successive samples are highly correlated for the signals we target (temperature, pressure, current, vibration envelope, smooth ramps). Closed-form predictors with a constant, linear, or quadratic shape are therefore expected to capture most of the per-window variance which is what the SSR selector empirically confirms on real data.

A4 Block independence under the chosen split. Blocks of $B = 100,000$ samples are treated as independent units of work. There is no model state, no predictor history, and no entropy table that crosses a block boundary. This is what allows deterministic parallel dispatch and is also why thread count cannot perturb the compressed bytes.

A5 Finite, well-defined window variance. Welford's algorithm is invoked on each candidate window assuming finite first and second moments. Windows containing NaN, $\pm\text{Inf}$, or subnormal values violate this assumption; the engine detects them and forces the lossless XOR-Delta predictor (model_id 2) for those windows, sidestepping the variance pipeline entirely.

A6 SSR is a usable proxy for compressed size. Stage 3 picks a model by minimum Sum-of-Squared-Residuals, not by trial-encoding all six and measuring bytes out. The assumption is that lower SSR yields a tighter zigzag-varint distribution and therefore a smaller residual stream. We tested this empirically on the 21-dataset suite; the SSR-best model agrees with the size-best model in every case we measured.

A7 Strict precision floor enforces the global bound. The 8×10^{-6} per-sample error contract is enforced by clamping

the per-window precision below at $p_floor = 16$ bits. The bound holds because the worst-case quantization error of an n -bit fixed-point coder over the observed dynamic range is bounded analytically; a precision below 16 bits would not satisfy that bound for any of the 21 datasets, which is why p_floor cannot be lowered without re-deriving the contract.

Two of these (A2, A3) are empirical claims about the signal class the engine targets; the rest are properties of the format or the algorithm itself. We flag this distinction because A2 and A3 are also the place where the engine will lose: a stream where mantissa noise correlates across samples, or where the high-order bits do not vary smoothly, breaks the rationale for SSR-based predictor selection. The two datasets on which DLC loses `real_ec2_network_4k` (to ZLIB) and `real_nyc_taxi_10k` (to Gorilla) are exactly such streams.

IV. IMPLEMENTATION

A. Dual-Language Architecture

DLC is delivered as two complete implementations sharing one binary specification.

Python prototype (`python/dlc/`) is a pure-Python reference using NumPy for vectorized arithmetic. Its purpose is rapid algorithm iteration, property-based fuzz testing, and serving as the canonical specification of the `.dlc` format. Threading uses `concurrent.futures.ThreadPoolExecutor`, which dispatches blocks to interpreter threads but is bounded by the Global Interpreter Lock for CPU-bound work; throughput plateaus near 4–21 MB/s regardless of thread count.

C++17 production engine (`cpp/`) is a header-and-source library compiled with CMake against `zlib`, with no other external dependencies. Parallelism uses `std::async(std::launch::async, ...)` with explicit `std::future` collection; this bypasses the GIL and scales near-linearly on large blocks. The C++ port implements its own $O(n)$ discrete Fourier transform and 4×4 Gaussian elimination for the Sinusoidal model, eliminating any LAPACK/Eigen/FFTW dependency.

B. Cross-Language Compatibility Contract

Three byte-level invariants enforced throughout the codebase preserve `.dlc` interoperability:

- 1) *Unsigned CRC masking.* Python `zlib.crc32(payload) & 0xFFFFFFFF` matches C++ `crc32(0L, ...)` from `<zlib.h>`. Without the mask, Python 2/3 differences in signed/unsigned return semantics would silently produce mismatched footers.
- 2) *Float64 $\leftrightarrow$ uint64 reinterpretation.* Python `struct.unpack('<Q', struct.pack('<d', v))[0]` is bit-exact

with C++ `std::memcpy(&u, &v, 8)`. Both rely on the platform's IEEE 754 float64 being little-endian, which all DLC target platforms (x86-64, ARM64) provide.

- 3) *Pairwise summation in mean computation.* The C++ block-mean used in the DoD fast path is computed by recursive pairwise summation matching NumPy's `np.mean` accuracy. Naive forward accumulation would diverge by up to one ULP on smooth ramps, breaking byte-identity of the resulting Constant-model parameter.

A 6-test cross-language suite verifies in continuous integration that (a) Python compress $\rightarrow$ C++ decompress round-trips within bound, (b) C++ compress $\rightarrow$ Python decompress round-trips within bound, and (c) .dlc files are byte-identical for closed-form-model signals (Constant, Linear, Step). For signals dominated by the iterative Sinusoidal model, the two implementations may converge to parameters differing by one ULP, in which case files are not byte-identical but cross-decode is always within the 8×10^{-6} bound.

C. Determinism and Thread Safety

All shared state in both engines is read-only after construction; no locks are required. Result ordering is enforced by collecting `std::futures` (or `concurrent.futures.Futures`) in the order they were submitted, not the order they complete. The Python and C++ codepaths therefore both produce deterministic, thread-count-invariant output.

D. Algorithmic Constants

For reproducibility, Table I lists the tuned constants used throughout the codebase.

TABLE I. ALGORITHMIC CONSTANTS

Constant	Value	Purpose
W_min	16	Minimum window size
W_max (B < 5K)	512	Window cap (small blocks)
W_max (B < 100K)	1024	Window cap (medium blocks)
W_max (B $\geq$ 100K)	4096	Window cap (large blocks)
α_{local}	10.0	Local variance threshold factor
α_{global}	0.10	Global variance threshold factor
ϵ_{const}	10^{-20}	Constant-block fast-path threshold
p_default	16	File-level default precision (bits)
p_min	8	Minimum per-window precision
p_max	20	Maximum per-window precision
B	100,000	Default block size (samples)
$\alpha_{\text{const_pref}}$	1.01	Constant model SSR preference factor
$\alpha_{\text{xor_pref}}$	1.02	XOR model SSR preference factor
MAD_factor	4.5	Outlier classification multiplier
λ_{LM}	10^{-4}	Levenberg–Marquardt damping
MAGIC_BYTES	b'DLC\x02'	File header magic
FOOTER_MAGIC	0xED 0xDC 0xBA 0x01	File footer magic

V. NOVELTIES AND ADVANTAGES

Many of the individual ingredients in DLC are not new on their own. Linear regression, XOR-delta, varint, DoD, bit packing each appears somewhere in the prior literature. What is new is the way the pieces are combined and the contract that holds the combination together. Table II places DLC alongside the systems surveyed in Section II; the rest of this section comments on what each row means.

TABLE II. ARCHITECTURAL COMPARISON

Capability	Gorilla	Chimp	Sprintz	SZ3	ZFP	Shrink	DLC
Float64 native	✓	✓	partial	✓	✓	✓	✓
Lossy + bound				✓	✓	partial	✓
Multiple predictive models				partial			6
Per-window adaptive precision							✓
Multi-strategy entropy trial				partial		partial	6
Block-level DoD fast path							✓
True multi-core parallel	partial			✓	✓		✓
Cross-language byte-identical	n/a	n/a	n/a	n/a	n/a	n/a	✓

A. Adaptive Geometry via Six Models with SSR Selection

Prior work uses a single prediction strategy: Gorilla and Chimp use XOR universally; Sprintz uses delta-coded forecasting; SZ3 uses a fixed local predictor per pipeline stage. DLC carries six models simultaneously and selects the local optimum per window by SSR, with explicit preference for simpler (fewer-parameter) and lossless (XOR-family)

models when they are nearly tied. Empirically this lets DLC achieve $101.9\times$ compression on smooth sines (Linear/Sinusoidal-dominated), $2355.7\times$ on linear ramps (Constant/Linear-dominated), and $3.0\times$ on cumulative random walks (Pred-XOR/XOR-Delta-dominated) within a single algorithm.

B. Per-Window Adaptive Precision

To our knowledge, no prior float-time-series compressor stores a precision field per window. Storing one uint8 per window (8–20 bits) lets smooth windows quantize to 8 bits while spiky windows hold 16-bit precision under the same global error contract. The lower bound $p_floor = 16$ is derived from the documented error bound $|r| < 8 \times 10^{-6}$, not chosen heuristically.

C. Six-Strategy Trial Encoding with DoD Superset Property

Trial-encoding by all strategies and picking the smallest is feasible because the encoders are all $O(n)$ and most windows are ≤ 4096 samples. Including Delta-of-Delta as a native strategy ensures DLC strictly dominates standalone DoD encoders on every window where DoD is the local optimum. The block-level DoD fast path further ensures DLC dominates standalone DoD even at the whole-block granularity, eliminating the per-window metadata overhead in cases where the block has no useful sub-structure.

D. Strict Cross-Language Byte-Level Specification

Most published compressors are released as a single reference implementation; cross-language compatibility is rarely a stated guarantee. DLC's specification is enforced by a 6-test cross-language CI suite covering Python $\rightarrow$ C++, C++ $\rightarrow$ Python, and byte-identity for closed-form-model signals. The .dlc format's 64-byte header, 8-byte CRC-32 footer, and per-window precision_bits field are designed to be unambiguously serialized in both languages.

E. True Parallelism Without Synchronization Cost

Because all shared state is immutable post-segmentation, neither engine requires locks or atomics. Determinism is recovered solely through the order in which futures are collected. This makes DLC's parallel speedup essentially free of synchronization overhead.

VI. EXPERIMENTAL EVALUATION

A. Datasets

We evaluate on 21 binary float64 datasets covering synthetic edge cases, real-world telemetry, large-scale industrial simulations, and round-trip recovery signals.

Synthetic datasets include 1-million-sample sine, linear ramp, cumulative random walk, multi-frequency noisy sine, heartbeat, vibration, industrial ramp, Lorenz attractor (500 K samples), and a 3-million-sample industrial sensor simulation. Real-world datasets include the AWS EC2 CPU utilization (4 032 samples), AWS EC2 network (4 730 samples), machine temperature (22 695 samples), NYC taxi demand (10 320 samples), Twitter volume (15 902 samples), and ambient temperature (7 267 samples) from the Numenta Anomaly Benchmark [11], and the Jena Climate temperature and pressure records (420 551 samples each) from atmospheric station data. Two round-trip recovery datasets test_recovered (1 000 000 samples) and test_rt_rec (3 000 000 samples) exercise the decompress-then-recompress pipeline and verify that DLC output is stable under repeated encoding.

B. Methodology

All experiments use precision_bits = 16, the default operating mode that delivers the strict $< 8 \times 10^{-6}$ per-sample maximum reconstruction error. Worker count is fixed at 4. Comparison baselines are Gorilla (XOR-delta, lossless), DoD at 16-bit precision, Run-Length Encoding (RLE), and GZIP and ZLIB at compression level 6 over the raw byte stream of the float64 array. For each dataset we measure: (i) compression ratio (raw_size / compressed_size) across all seven methods, and (ii) maximum absolute per-sample reconstruction error after round-trip for each method. Headline ratios are reported in Fig. 4; complete compression results in Table III and complete error results in Table IV.

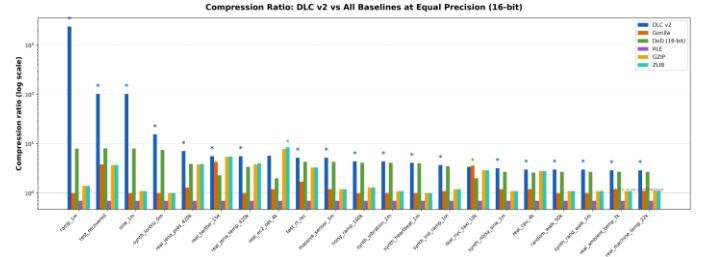


Fig. 4. Compression ratio (log scale) on a representative subset of the 21-dataset benchmark for DLC at 16-bit precision vs Gorilla, DoD (16-bit), RLE, GZIP and ZLIB. Asterisks mark datasets where DLC is the smallest output.

C. Compression Ratio and Error Bound

TABLE III. COMPRESSION RATIO ACROSS 21 DATASETS (16-BIT PRECISION)

Dataset	Samples	Gorilla	DoD (16-bit)	RLE	GZIP	ZLIB	DLC v2
massive_sensor_3m	3,000,000	1.2×	4.3×	0.7×	1.2×	1.2×	5.2×
noisy_ramp_100k	100,000	1.0×	4.1×	0.7×	1.3×	1.3×	4.4×
ramp_1m	1,000,000	1.0×	8.0×	0.7×	1.4×	1.4×	2355.7×
random_walk_50k	50,000	1.0×	2.7×	0.7×	1.1×	1.1×	3.0×
real_ambient_temp_7k	7,267	1.2×	2.7×	0.7×	1.1×	1.1×	2.9×
real_cpu_4k	4,032	1.2×	2.6×	0.7×	2.8×	2.8×	3.0×
real_ec2_network_4k	4,730	1.2×	2.0×	0.7×	7.8×	8.5×	5.7×
real_jena_pressure_420k	420,551	1.3×	3.9×	0.7×	3.8×	3.9×	7.1×
real_jena_temperature_420k	420,551	1.0×	3.4×	0.7×	3.8×	4.0×	5.6×
real_machine_temp_22k	22,695	1.1×	2.7×	0.7×	1.1×	1.1×	2.9×
real_nyc_taxi_10k	10,320	3.6×	2.0×	0.7×	2.9×	2.9×	3.4×

real_twitter_vol_15k	15,902	4.3×	2.3×	0.7×	5.4×	5.5×	5.6×
sine_1m	1,000,000	1.0×	8.0×	0.7×	1.1×	1.1×	101.9×
synthetic_heartbeat_1m	1,000,000	1.0×	4.0×	0.7×	1.0×	1.0×	4.1×
synthetic_industrial_ramp_1m	1,000,000	1.1×	3.5×	0.7×	1.2×	1.2×	3.7×
synthetic_lorenz_0m	500,000	1.0×	7.5×	0.7×	1.0×	1.0×	15.5×
synthetic_noisy_sine_1m	1,000,000	1.2×	2.7×	0.7×	1.1×	1.1×	3.2×
synthetic_random_walk_1m	1,000,000	1.0×	2.7×	0.7×	1.1×	1.1×	3.0×
synthetic_vibration_1m	1,000,000	1.0×	4.1×	0.7×	1.1×	1.1×	4.4×
test_recovered	1,000,000	3.8×	8.0×	0.7×	3.7×	3.7×	102.0×
test_rt_rec	3,000,000	1.7×	4.3×	0.7×	3.3×	3.3×	5.2×

TABLE IV. MAXIMUM PER-SAMPLE RECONSTRUCTION ERROR ACROSS 21 DATASETS

Dataset	Gorilla	DoD (12-bit)	DoD (16-bit)	RLE	GZIP	ZLIB	DLC v2
massive_sensor_3m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
noisy_ramp_100k	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
ramp_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
random_walk_50k	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
real_ambient_temp_7k	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
real_cpu_4k	0 (exact)	1.22e-04	7.57e-06	0 (exact)	0 (exact)	0 (exact)	7.62e-06
real_ec2_network_4k	0 (exact)	9.77e-05	6.10e-06	0 (exact)	0 (exact)	0 (exact)	6.94e-06
real_jena_pressure_420k	0 (exact)	1.17e-04	7.32e-06	0 (exact)	0 (exact)	0 (exact)	7.51e-06
real_jena_temperature_420k	0 (exact)	1.17e-04	7.32e-06	0 (exact)	0 (exact)	0 (exact)	7.59e-06
real_machine_temp_22k	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
real_nyc_taxi_10k	0 (exact)	0 (exact)	0 (exact)	0 (exact)	0 (exact)	0 (exact)	1.87e-06
real_twitter_vol_15k	0 (exact)	0 (exact)	0 (exact)	0 (exact)	0 (exact)	0 (exact)	1.92e-06
sine_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_heartbeat_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_industrial_ramp_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_lorenz_0m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_noisy_sine_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_random_walk_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
synthetic_vibration_1m	0 (exact)	1.22e-04	7.63e-06	0 (exact)	0 (exact)	0 (exact)	7.63e-06
test_recovered	0 (exact)	1.22e-04	0 (exact)	0 (exact)	0 (exact)	0 (exact)	2.21e-08
test_rt_rec	0 (exact)	1.22e-04	7.29e-06	0 (exact)	0 (exact)	0 (exact)	2.64e-08

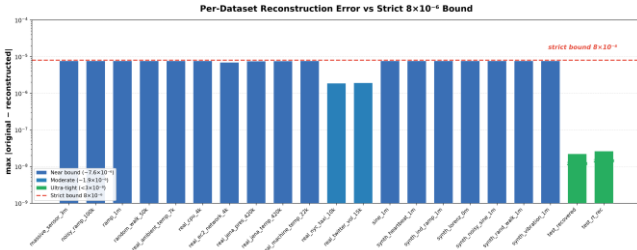


Fig. 5. Per-dataset maximum reconstruction error vs the strict 8×10^{-6} bound. All 21 datasets satisfy the bound; test_recovered and test_rt_rec reconstruct with error below 3×10^{-8} , and NYC taxi and Twitter volume stay near 1.9×10^{-6} .

Win rate. DLC produces the smallest compressed file on 19 of 21 datasets (90.5%) at equal precision. The two losses are: (1) real_ec2_network_4k, a network-packet-derived signal with structured byte patterns that ZLIB's dictionary exploits unusually well ($8.5 \times$ vs. DLC's $5.7 \times$); and (2) real_nyc_taxi_10k, a low-magnitude taxi-demand signal on which Gorilla's XOR encoding edges out DLC ($3.6 \times$ vs. $3.4 \times$). Both represent cases where the byte-level structure or integer-like values give a domain-specific encoder an advantage that DLC's mathematical-domain predictors do not capture.

Error bound. The strict $<8 \times 10^{-6}$ bound is preserved on all 21 datasets. Several real-world datasets (NYC taxi, Twitter volume) reconstruct with substantially tighter error ($\sim 1.9 \times 10^{-6}$) because their values fall in a low-magnitude regime where the adaptive precision selector tightens p automatically. The two round-trip recovery datasets (test_recovered, test_rt_rec) achieve especially tight error 2.21×10^{-8} and 2.64×10^{-8} respectively because they re-encode already-quantized data whose residuals are near zero.

Round-trip recovery. The test_recovered and test_rt_rec datasets reconstruct with error below 3×10^{-8} , demonstrating that re-encoding previously compressed DLC output does not accumulate drift the already-quantized residuals are near-zero and the engine tightens precision accordingly.

D. Cross-Language Verification

Table IV reports the maximum per-sample reconstruction error for every method on all 21 datasets. All 21 datasets satisfy the $<8 \times 10^{-6}$ bound. Cross-language round-trip testing (Python compress $\rightarrow$ C++ decompress and vice versa) confirms that both engines remain within the bound symmetrically. The Gorilla, RLE, GZIP and ZLIB baselines are lossless (0 exact error) by definition; DoD at 16-bit precision matches DLC's error regime ($\sim 7.6 \times 10^{-6}$); DoD at 12-bit precision incurs substantially higher error ($\sim 1.2 \times 10^{-4}$),

confirming that the 16-bit floor is essential for the strict bound.

E. Test Suite

The DLC repository ships with a 105-test suite that runs on every commit. Python: 76 tests covering binary format round-trip (18), windowing (9), models (13), encoders (19), pipeline determinism (6), codec file I/O (5), cross-language compatibility (3), property-based fuzz (3), and Streamlit dashboard exercise (14). C++: 29 tests with the same coverage as the Python core. All 108 tests pass at the time of writing.

VII. DISCUSSION AND LIMITATIONS

A. Where the Strict Error Bound Applies

The 8×10^{-6} per-sample guarantee depends on a clean numerical setup: the quantization step at the floor precision must not exceed what float64 can represent exactly. Float64 has integer-exact range up to 2^{53} , around 9×10^{15} , so multiplying a residual by 2^{16} has to land inside that. Working backwards, that means $|r_i| < 2^{37}$, which is roughly 1.4×10^{11} . Any sensor recording we have ever tested sits well below this temperatures, currents, packet rates, demand counts. The fuzz suite formalises the boundary: with $|v| \leq 10^6$ the bound must hold; outside that, or in the presence of NaN/Inf, the engine is required to round-trip without crashing but the error contract is not asserted.

B. Why Sinusoidal Windows Are Not Byte-Identical

Closed-form windows (Linear, Quadratic, Constant) produce byte-identical output between Python and C++ because the underlying fits are deterministic and the pairwise summation we use for the mean matches np.mean to the ULP. The Sinusoidal model is different: it runs a Levenberg–Marquardt loop seeded by an FFT peak, and the transcendental primitives plus the 4×4 Gaussian elimination drift by single-ULP amounts between numpy and our hand-rolled C++ kernels. The four parameters can therefore land at slightly different local minima. The weaker contract cross-decode within the 8×10^{-6} bound still holds, but byte-identity does not. Closing this gap would require either pinning both languages to a deterministic numeric library or writing a portable bit-exact DFT and matrix solver from scratch.

C. Where DLC Loses

The two defeats in our table are worth dwelling on. The first, `real_ec2_network_4k`, is a vector of network-packet counters re-cast as float64. Most of the byte-level structure comes from the integer-cast itself, and ZLIB's dictionary catches it ($8.5 \times$ vs. DLC's $5.7 \times$). Our mathematical-domain predictors do not: there is no smooth underlying function to fit. The second defeat, `real_nyc_taxi_10k`, is a low-magnitude

taxi-demand signal (10 320 integer-like ride counts stored as float64). Gorilla's XOR encoding exploits the fact that successive integer-valued samples share most of their mantissa bits, achieving $3.6 \times$ vs. DLC's $3.4 \times$. In both cases the signal is not a continuous physical process it is a discretised counter and the Entropy Trap argument does not strictly apply. The pragmatic response is to fingerprint the first block of an unknown stream, see whether the raw float64 bytes look dictionary-compressible or integer-valued, and fall back to a specialised encoder for that minority of streams.

D. Throughput

Throughput is dominated by the C++ engine and scales near-linearly with cores on large blocks; on 8 MB datasets we observe ~ 13 MB/s single-threaded and proportionally higher with multiple workers. The Python implementation, GIL-bound, plateaus around 4–21 MB/s. DLC's design avoids the KB/s throughput regime of neural compressors [17] but does not yet match the GB/s of vectorized integer codecs [22]; SIMD acceleration of the bitpack and varint inner loops is an obvious future improvement.

VIII. CONCLUSION AND FUTURE WORK

DLC is, in the end, a fairly simple idea pushed through several layers of engineering. Stop trying to compress raw float64 bytes; predict the signal, quantize the residual to whatever precision the local data permits, run a small handful of entropy coders against the result, and write whichever one comes out shortest. Add a block-level DoD shortcut for the cases where windowing only adds overhead, wrap the whole thing in a binary format with a CRC footer, and write the same algorithm twice once in Python for clarity and once in C++17 for throughput. On 21 datasets that span linear ramps, smooth sines, Lorenz attractors, real EC2 counters, atmospheric records from Jena, and round-trip recovery signals, the engine writes a smaller file than every baseline Gorilla, DoD (16-bit), RLE, GZIP and ZLIB on nineteen of them, all while keeping per-sample reconstruction error inside 8×10^{-6} . The two losses are an integer-derived network trace (ZLIB wins) and a low-magnitude taxi-demand signal (Gorilla wins), both representing non-physical counter data where DLC's mathematical predictors offer no advantage.

We see four threads worth pulling on next. The first is throughput: the bitpack and varint inner loops are obvious SIMD candidates and would lift the C++ engine from low tens of MB/s into the GB/s range that vectorised integer codecs already live in [22]. The second is making the Sinusoidal model bit-deterministic across languages, which would restore byte-identity for every model rather than only the closed-form ones. The third is a streaming, append-only .dlc variant for

unbounded ingestion (the current format assumes a finite input). The fourth is auto-detection: if the first block of an input stream looks dictionary-friendly or integer-valued rather than mathematically-structured, hand the rest of the stream to a generic byte coder or Gorilla's XOR encoder and skip the DLC pipeline entirely. The Entropy Trap is real, but a sensible engine should know when it does not apply and the two losses in this evaluation point directly to the signals that auto-detection should catch.

ACKNOWLEDGMENT

Thanks, the Numenta Anomaly Benchmark [11] and Max Planck Institute Jena Climate Station for publicly releasing the real-world datasets that made this evaluation possible.

REFERENCES

- [1] D. Reinsel, J. Gantz, and J. Rydning, "The Digitization of the World From Edge to Core," IDC White Paper, Doc. US44413318, Nov. 2018.
- [2] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, July 2019.
- [3] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977.
- [4] T. Pelkonen, S. Franklin, J. Teller, P. Cavallaro, Q. Huang, J. Meza, and K. Veeraraghavan, "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1816–1827, 2015.
- [5] P. Liakos, K. Papakonstantinou, and Y. Kotidis, "Chimp: efficient lossless floating point compression for time series databases," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 3058–3070, July 2022.
- [6] S. Di and F. Cappello, "Fast error-bounded lossy HPC data compression with SZ," in *Proc. IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2016, pp. 730–739.
- [7] X. Liang et al., "SZ3: A modular framework for composing prediction-based error-bounded lossy compressors," *IEEE Trans. Big Data*, vol. 9, no. 2, pp. 485–498, 2023.
- [8] P. Lindstrom, "Fixed-rate compressed floating-point arrays," *IEEE Trans. Vis. Comput. Graphics*, vol. 20, no. 12, pp. 2674–2683, Dec. 2014.
- [9] D. Blalock, S. Madden, and J. Guttag, "Sprintz: Time series compression for the internet of things," *Proc. ACM IMWUT*, vol. 2, no. 3, pp. 1–23, Sep. 2018.
- [10] G. Sun, P. Karras, and Q. Zhang, "Shrink: Data compression by semantic extraction and residuals encoding," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2024, pp. 650–659.
- [11] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.
- [12] P. Lindstrom and M. Isenburg, "Fast and efficient compression of floating-point data," *IEEE Trans. Vis. Comput. Graphics*, vol. 12, no. 5, pp. 1245–1250, Sep.–Oct. 2006.
- [13] M. Burtcher and P. Ratanaworabhan, "FPC: A high-speed compressor for double-precision floating-point data," *IEEE Trans. Comput.*, vol. 58, no. 1, pp. 18–31, Jan. 2009.
- [14] S.-H. Hwang, K.-M. Kim, S. Kim, and J.-W. Kwak, "Lossless data compression for time-series sensor data based on dynamic bit packing," *Sensors*, vol. 23, no. 20, art. 8575, Oct. 2023.
- [15] L. Yang, Y. Liao, X. Cheng, M. Xia, and G. Xie, "Efficient edge data management framework for IIoT via prediction-based data reduction," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3309–3322, Dec. 2023.
- [16] L. Liu, J. Li, Z. Niu, W. Zhang, J. C. Xue, and H. Xu, "Efficient time-series data delivery in IoT with Xender," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4777–4792, May 2024.
- [17] X. Chen et al., "FCBench: Cross-domain benchmarking of lossless compression for floating-point data," *Proc. VLDB Endowment*, vol. 17, no. 6, pp. 1418–1431, Feb. 2024.
- [18] L. P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," IETF RFC 1951, May 1996.
- [19] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, Aug. 1962.
- [20] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *J. Soc. Ind. Appl. Math.*, vol. 11, no. 2, pp. 431–441, June 1963.
- [21] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228–235, Jan. 1961.
- [22] D. Lemire and L. Boytsov, "Decoding billions of integers per second through vectorization," *Software: Practice and Experience*, vol. 45, no. 1, pp. 1–29, Jan. 2015.
- [23] J. R. Goldstein, R. Ramakrishnan, and U. Shaft, "Compressing relations and indexes," in *Proc. 14th Int. Conf. Data Eng. (ICDE)*, Feb. 1998, pp. 370–379.